Recovery Compression of Cross-Support Spread in 2:4 LLM Sparsification: An Archived 0.118 pp Observation

Anonymous authors
Paper under double-blind review

Abstract

Exact 2:4 sparsity couples support decisions locally, while post-training recovery can obscure their downstream differences. We examine three registered fixed supports—ALPS, ELSA-4096, and ProxSparse—after the same 625M-token joint sparse-weight and rank-16 SLoRB recovery. Their cross-support Avg-9 point-estimate range contracts from 2.675 pp before recovery to 0.118 pp afterward. The latter is smaller than the harness's 0.52 pp marginal binomial standard error for a single checkpoint, but that quantity is not a pairwise or joint resolution limit. Per-item predictions were not retained, so the joint uncertainty of the post-recovery range and residual support differences are unidentified. The result is therefore an archived descriptive observation and a measurement caution: claims at this scale require paired per-item analysis. The curvature score, annealing schedule, and SLoRB branch remain pipeline background because they were not isolated under matched budgets. An archived 5B checkpoint provides supporting context only: it is evaluated with SLoRB active, has no folded exact-2:4 export or pre/post-export comparison, and its recovery-corpus manifest is unavailable.

1 Introduction

Semi-structured sparsity is one of the few compression regimes that combines substantial parameter removal with native accelerator support. In particular, NVIDIA sparse Tensor Cores execute exact 2:4 patterns, retaining two weights in every group of four (NVIDIA, 2020). The same constraint that makes deployment efficient also couples pruning decisions: each candidate competes locally, and a mistaken hard selection cannot be repaired without changing the support.

We ask a narrower empirical question: if several native supports undergo an *identical* token budget and recovery recipe, does their cross-support spread persist? The comparisons summarized in Section 2 do not isolate this before/after quantity across multiple labelled supports. Our load-bearing contribution is the archived measurement of that quantity; the SparseForge pipeline below is background context, not a validated mechanism.

Calibration-only methods such as Wanda and SparseGPT (Sun et al., 2023; Frantar & Alistarh, 2023) are inexpensive, while retraining-based AST and CAST (Huang et al., 2024; 2025) recover more quality at larger token budgets. This leaves a practical question: can a recovery run spend its tokens on improving the structured support itself, rather than merely adapting weights to an early hard mask?

As pipeline background, SparseForge (Figure 1) maintains a continuous mask and alternates task/distillation updates of the sparse base weights and rank-16 SLoRB branch with periodic curvature-score mask updates. A soft-to-hard schedule then moves the mask toward an exact 2:4 pattern. This describes the archived training pipeline rather than an isolated causal mechanism: the reported checkpoint keeps the SLoRB branch live, and folding it into a deployable exact-2:4 export is a registered future step (Appendix C), not an executed result. These three components have never been isolated under matched budgets; none of the reported gains can be attributed to any single mechanism.

The recovery-compression comparison in Table 5 is the central result. The three registered supports span 2.675 pp in Avg-9 before recovery and 0.118 pp after the same 625M-token recov-

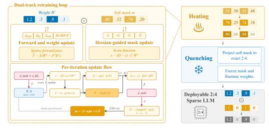

Figure 1: **SparseForge pipeline background.** The weight track updates sparse base weights W and rank-16 SLoRB parameter B using task and distillation losses. A separate periodic mask track computes curvature-based importance, constructs a soft group gate, and anneals it toward an exact 2:4 target. Quenching freezes the support; the dashed fold/re-projection step depicted as the export arrow is the planned branch fold and re-projection, which is shown for completeness and is *not executed* by the frozen finalizer—this paper reports only the branch-active training-state checkpoint upstream of that boundary. The regularizer $\mathcal{L}_{\text{mid}}$ affects the dedicated mask update, not the weight-loss autograd path. The diagram describes the pipeline and is not evidence that any component causes the reported result. The in-figure annotations are set below body-text size so that the weight track, the mask track, and quenching fit one column width; the key symbolic annotations (s, G, m, $\widehat{Y}$) follow the macro naming of Sec. 4.

ery. The latter is smaller than the harness's 0.52 pp marginal single-checkpoint standard error, but that scale is not a threshold, detection floor, or equivalence margin. Without paired per-item predictions, separability of the three supports remains untested. The archived 5B comparison in Table 1 and Figure 2 is supporting context rather than mechanism evidence. We contribute:

- an archived observation that identical recovery compresses cross-support Avg-9 spread from 2.675 to 0.118 pp;
- an explicit requirement for paired per-item analysis and joint bounds before comparing the recovered supports at this scale; and
- a unified local evaluation that separates fixed-support recovery from an archived branch-active checkpoint and records its corpus and export boundaries.

2 Related Work

One-shot and calibration-scale pruning. Magnitude pruning and second-order criteria established the basic importance-estimation view (Han et al., 2015; LeCun et al., 1990; Hassibi & Stork, 1993). For LLMs, SparseGPT uses approximate second-order information and Wanda combines weights with activation statistics (Frantar & Alistarh, 2023; Sun et al., 2023). ALPS performs layerwise constrained reconstruction (Meng et al., 2024); ELSA and ProxSparse optimize semi-structured supports or parameters at calibration scale (Lee et al., 2026; Liu et al., 2025). These approaches avoid a large recovery corpus, but their support decisions remain consequential under exact local structure.

Semi-structured sparse training. MaskLLM learns discrete $N{:}M$ masks with a large corpus, so its mask is trained rather than frozen from a calibration decision (Fang et al., 2024). AST anneals dense weights toward 2:4 and uses SLoRB for recovery (Huang et al., 2024); CAST uses a continuous differentiable relaxation (Huang et al., 2025). These approaches motivate learned structure, but the comparisons available to this study do not fix and label several native supports, apply the same recovery recipe to each, and measure the resulting cross-support spread. Our contribution is that archived measurement rather than a novel algorithmic advantage. Because the SparseForge mask

trajectory was not isolated under matched budgets, the paper makes no component-level mechanism claim.

3 Pipeline Rationale (Background)

In a group of four weights, one incorrect survivor necessarily excludes another candidate. A one-shot score can therefore select a poor subspace before recovery begins. Keeping the mask soft allows repeated evidence to reverse early rankings, but a soft gate is not deployable. The central problem is consequently a constrained transition: preserve exploration early, inject exact group structure progressively, and minimize the final soft-to-hard projection gap.

Curvature gives a local survival signal. For a weight w_i near a pretrained solution,

$$\Delta L_i \approx -\frac{\partial L}{\partial w_i} w_i + \frac{1}{2}\frac{\partial^2 L}{\partial w_i^2} w_i^2. \tag{1}$$

When the first-order term is small, magnitude alone misses the loss curvature associated with deletion. Optimal Brain Damage motivates second-order importance via a *diagonal* Hessian proxy (LeCun et al., 1990), while Optimal Brain Surgeon and WoodFisher go beyond the diagonal precisely because cross-term and inverse-curvature effects matter under exact structure (Hassibi & Stork, 1993; Singh & Alistarh, 2020): for $N{:}M$ groups the deleting mask couples coordinates within a group, so the diagonal term is only a first-order faithful ranking signal, not a certificate. SparseForge therefore uses the diagonal curvature score to rank *candidates* within each structured group (the cheap sufficient signal for which column of a 2:4 group to drop) and makes no stronger approximation claim than OBD; annealing, not the curvature proxy alone, is what introduces the final support gradually. Throughout this paper we use "match" in the sense of numerical equivalence within stated sampling error, not of verified behavioral equivalence between checkpoints; per-item agreement between SparseForge and AST has not been established in this workspace.

4 Training Pipeline

For reproducibility, we describe the archived training pipeline that produced the registered endpoints; the description is background and does not isolate component effects. Let a linear layer be $Z = XW^\top$, with $W \in \mathbb{R}^{D\times C}$, and let $m \in [0,1]^{D\times C}$ be a soft mask. The effective sparse weight is $\widetilde{W} = m \odot W$. For a fixed projection P and learnable rank-16 SLoRB matrix B, the reported strong configuration computes

$$\widehat{Y} = X(\widetilde{W}^\top + P^\top B^\top). \tag{2}$$

4.1 Reported Weight and Mask Tracks

The weight track minimizes next-token cross-entropy plus teacher-to-student KL divergence,

$$\mathcal{L}_{W,B} = \lambda_{\text{task}}\mathcal{L}_{\text{task}} + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}}, \tag{3}$$

updating both W and B. The mask is a non-autograd buffer with a dedicated periodic rule. This separation permits frequent weight adaptation while avoiding a noisy mask decision at every minibatch. It also makes the method explicitly joint: the support, sparse base weights, and recovery branch all change during the strong run.

4.2 Curvature-Score Background

For each layer, we estimate diagonal curvature with a Hutchinson-style probe (Hutchinson, 1990) and maintain an exponential moving average. The survival score is

$$s = \Big(\max(H, 0) + \epsilon\Big) \odot W^2. \tag{4}$$

where H is the Hutchinson-estimated diagonal and the element-wise clamp at zero prevents sign reversals from mis-estimated negative diagonal entries from re-ordering the within-group ranking. After layer-wise standardization, each non-overlapping group g of M entries forms a soft gate and a structured target,

$$G_g = \sigma((\widetilde{s}_g - \tau_g)/T), \qquad \bar{G}_g = \mathrm{TopK}_N(\widetilde{s}_g), \tag{5}$$

where τ_g is the midpoint between the Nth- and $(N{+}1)$th-largest standardized scores; the frozen implementation instead uses the Nth-largest score itself, so the Nth-ranked candidate sits exactly at threshold rather than strictly above it (Appendix C). T controls decisiveness without immediately committing to a binary support.

4.3 Soft-to-Hard Schedule and Deployment Boundary

We progressively mix the exploratory gate with its feasible target,

$$G \leftarrow (1-\beta)G + \beta\bar{G}, \tag{6}$$

while decreasing T and ramping a mild binary-target penalty $\mathcal{L}_{\text{mid}} = \|m - \bar{G}\|_F^2/(2|m|)$. The dedicated mask update moves a clipped gate toward $\bar{G}$ and then applies an exponential moving average. Near the end, an interpolation $m_{\text{eff}} = xm + (1-x)1[m > \theta]$ moves from the soft mask to the binary mask. The scheduled construction would then fold the branch into the weights and re-apply exact 2:4 projection. This is a scheduled construction, not a theoretical convergence guarantee. In the frozen implementation the schedule stops at the branch-active checkpoint: the branch fold and exact 2:4 re-projection are specified but not executed (Appendix C), so every downstream number is a training-state measurement. All hyperparameters of the reported strong configuration are collected in Table 6 (Appendix C).

4.4 Notation

Throughout, SLoRB denotes the **S**parse-**Lo**w-**R**ank **B**ranch (Huang et al., 2024): a fixed projection P and learnable rank-16 matrix B whose product BP participates in every training-state evaluation; the algebraic fold of BP into the weights at quenching time is specified but not implemented by the frozen finalizer (Appendix C), and is registered as future export work.

5 Experiments

5.1 Setup

The primary study uses LLaMA-2-7B (Touvron et al., 2023) at exact 2:4 sparsity. The main checkpoint performs 5B-token recovery; its training corpus provenance is unavailable (the archived manifest lives with the checkpoint on a host unreachable from this workspace; Appendix C), so the corpus is not named in this paper. We evaluate zero-shot plain accuracy with lm-evaluation-harness (Biderman et al., 2024) on BoolQ, RTE, HellaSwag, WinoGrande, ARC-Easy, ARC-Challenge, OpenBookQA, RACE, and PIQA (Clark et al., 2019; Wang et al., 2019; Zellers et al., 2019; Sakaguchi et al., 2019; Clark et al., 2018; Mihaylov et al., 2018; Lai et al., 2017; Bisk et al., 2019). Avg-9 is the task-equally-weighted nine-task aggregate (unweighted mean of the nine per-task accuracies; each task contributes $1/9$ regardless of its dev-set size). We also report WikiText-2 perplexity. Every reported evaluation, including the headline row, is of the branch-active training-state checkpoint (SLoRB branch live at inference); no folded or re-projected export exists, and export-equivalence is prospective (Appendix C).

Table 1 separates native calibration/short-optimization checkpoints from fixed-support recovery. Each recovery row fixes the named support and jointly trains sparse base weights plus zero-initialized rank-16 SLoRB for 624,951,296 tokens (reported as 625M). The public AST export and the archived SparseForge checkpoint are evaluated locally; the AST publication does not disclose its own recovery-token budget or recovery corpus, so the AST row is a quality anchor rather than a matched-budget control. The 5B comparison is a single-checkpoint point estimate; historical repeated runs are confined to Appendix E. For the pending matched-budget arm we tokenized a 1.366B-token Dolmino-mix-1124 recovery stream locally, and its validation split is an 8192-token prefix of that same training stream (`val = train[:8192]`); it serves only as an early-step sanity signal for perplexity stability, not as an independent held-out set, so no generalization claim is drawn from it.

[1] The 625M matched-budget arm is registered as a pending measurement.

[2] The recovery corpus of this archived 5B checkpoint is not named because its provenance is unverifiable in this workspace: the training manifest and checkpoint live on an unreachable archive host (Appendix C); the pending matched-budget arm tokenizes a locally controlled Dolmino-mix-1124 stream instead.

Table 1: Unified LLaMA-2-7B evaluation at exact 2:4 sparsity. Task cells are local plain-accuracy percentages; PPL is WikiText-2. Wanda, SparseGPT, ALPS, and ELSA are three-seed means. Recovery rows are not native checkpoints. The 625M SparseForge fixed-support matched-budget arm[1] and the recovery token budget and corpus of the public AST checkpoint are registered gaps; the AST row is a quality anchor with under-reported recovery-budget provenance, not a matched-budget control. This aggregate supports the archived AST comparison; Appendix F, Table 9 reports baseline dispersion. Task abbreviations: BQ = BoolQ, Hella. = HellaSwag, Wino. = WinoGrande, ARC-e/c = ARC-Easy/Challenge, OBQA = OpenBookQA. The SparseForge row is a branch-active training-state checkpoint (SLoRB live at inference); a folded exact-2:4 export has never been produced or measured (Appendix C, EXT-M3). The SparseForge row is additionally a 5B-token recovery checkpoint whose training-corpus provenance is unavailable (archived with the checkpoint; Appendix C); only the pending matched-budget arms use a locally tokenized Dolmino-mix-1124 stream.

(a) Per-task accuracy: BoolQ through ARC-Easy

Method	BQ	RTE	Hella.	Wino.	ARC-e
Dense reference	79.45	62.82	57.10	69.61	75.55
Wanda (3 seeds)	68.08	53.07	41.45	61.64	61.73
SparseGPT (3 seeds)	70.72	55.96	45.03	64.98	63.23
ALPS (3 seeds)	67.65	54.87	46.36	65.30	64.60
ELSA (4,096 steps; 3 seeds)	69.79	58.72	50.95	66.19	67.33
ProxSparse (official)	68.53	56.68	47.94	63.61	64.31
ALPS mask + recovery (625M)	69.72	62.45	52.60	67.09	69.53
ELSA mask + recovery (625M)	72.94	59.57	52.19	66.85	70.79
ProxSparse mask + recovery (625M)	71.96	61.73	52.12	66.61	70.37
AST official deployable	74.50	65.34	54.71	68.19	73.36
SparseForge (archived corpus[2], 5B)	78.47	49.82	54.65	69.85	76.39

(b) Per-task accuracy: ARC-Challenge through PIQA, with aggregate and PPL

Method	ARC-c	OBQA	RACE	PIQA	Avg-9	PPL
Dense reference	42.75	33.20	39.52	77.86	59.76	5.5637
Wanda (3 seeds)	29.78	23.47	35.95	70.37	49.50	12.3911
SparseGPT (3 seeds)	32.39	25.20	37.51	71.82	51.87	10.0569
ALPS (3 seeds)	32.14	26.13	38.50	73.56	52.12	10.3270
ELSA (4,096 steps; 3 seeds)	35.07	28.47	39.46	75.37	54.59	7.9832
ProxSparse (official)	33.53	25.00	35.79	71.87	51.92	9.2992
ALPS mask + recovery (625M)	35.58	28.00	40.96	76.44	55.82	7.1006
ELSA mask + recovery (625M)	36.52	28.60	39.81	76.17	55.94	7.1121
ProxSparse mask + recovery (625M)	36.18	28.00	39.90	76.28	55.91	7.1284
AST official deployable	39.51	30.00	38.85	76.99	57.94	6.3430
SparseForge (archived corpus[2], 5B)	43.86	35.20	41.63	76.33	**58.47**	**6.2179**

Under this harness, the per-checkpoint binomial-SE of Avg-9 is 0.52 pp (Appendix A.1); this is a single-checkpoint eval-sampling scale only, not a detection floor for pairwise or joint claims. The 0.52 pp term is a pre-declared harness quantity: it is the binomial standard error computed from the standard lm-eval dev-set sizes n_k listed with their tasks in Table 3 (derivation in Appendix A.1), fixed by the harness protocol and independent of the archived spread endpoints, so the marginal-scale comparisons above are not derived from the measurements they bound.

5.2 Recovery Compression as the Load-Bearing Result

The three native supports in Table 1 span 2.675 pp in Avg-9. Applying the same 625M-token joint sparse-weight and SLoRB recovery to each support narrows that range to 0.118 pp (Table 5), far below the 0.52 pp per-checkpoint binomial-SE sampling-noise scale. This is an archived descriptive observation: the identical recovery improves every fixed-support endpoint, yet the residual cross-support range cannot be tested against a pairwise or joint significance level from the registered

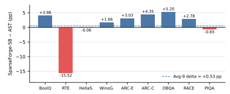

Figure 2: Per-task accuracy difference SparseForge-5B minus the public AST deployable checkpoint under the same nine-task harness, computed from the harness CSV. The Avg-9 difference is $+0.53$ points (dashed line), but task-level behavior is strongly mixed: BoolQ, ARC-Easy, ARC-Challenge, OpenBookQA, and RACE contribute the gain, while RTE moves by -15.52 points. The aggregate advantage over the strongest 625M fixed-support recovery endpoint is $+2.53$ points.

aggregate CSVs, because the per-item statistics needed for that test were not retained. It is therefore a historical endpoint fact, not a statistical negative result or equivalence claim.

Figure 2 retains the archived 5B comparison as supporting context. Its $+0.53$-point aggregate conceals strongly mixed task-level behavior: the gain is concentrated in BoolQ, ARC, OpenBookQA, and RACE, while RTE moves by -15.52 points against SparseForge. The aggregate remains within sampling noise and is too small to support broad superiority. It also does not establish a matched-budget SparseForge advantage because no valid 625M-token SparseForge endpoint exists in this protocol. Under validated provenance the per-task pattern could suggest a recovery-corpus composition effect, but that corpus is not verifiable in this workspace (Appendix C), so no corpus-composition attribution is claimed; the matched controls needed to separate these attributions remain pending (Sec. 7).

Table 2 separates the earlier deviating RTE log from the registered value and its denominator checks. The anomaly is consistent with a single-digit corruption (for example, a leading $4\to 6$ substitution), while denominator back-solving verifies every per-task harness cell against an integer example count. We therefore use the registered RTE value throughout. It does not drive the recovery-compression result: the majority-share Avg-9 margin remains close enough to zero that a small task-level change can reverse its sign.

Table 2: RTE provenance and denominator checks for the 5B checkpoint. The earlier 69.82 log is not representable as an integer correct count over 277 examples; the registered $49.82 = 138/277$ is used throughout. The $+0.528$-point Avg-9 margin remains sensitive to a small task-level change.

Check	Value	Interpretation
Earlier logged RTE cell	69.82	No integer numerator over $n = 277$ yields this percentage.
Registered RTE cell	49.82	Exact at 138/277; used in Table 1 and Figure 2.
RTE majority class	52.71%	Exact at 146/277; neither RTE value is near this baseline, and the registered value is below chance.
Same-row WinoGrande	69.85	Near-coincidence with the logged anomaly, consistent with a single-digit corruption.
Avg-9 margin	+0.528 pp	A change of $O(13)$ examples in one comparably weighted task can move it through zero.

RTE diagnostic. The registered RTE result is 49.82%, below the 52.71% majority-class baseline; Table 2 carries the provenance checks and the exact integer counts behind both rates. Neither a label-order nor a prompt-template ablation has been re-run in this workspace, so we cannot separate a harness artifact from a recovery-side task regression, and the AST row of Table 1 stays above chance on the same harness configuration. The cause therefore remains unresolved and registered: we

report the measured cell without using it as evidence for or against the method, and its leave-one-out consequence is isolated separately (Appendix A, Table 3).

5.3 What the Recovery Baselines Establish

The narrow post-recovery spread among ALPS-, ELSA-, and ProxSparse-supported endpoints shows that joint sparse-weight/SLoRB healing improves several fixed supports while leaving an observable range far below the harness's per-checkpoint binomial-SE sampling-noise scale. It does not prove that one native support is universally best, nor that recovery makes the supports intrinsically equivalent. A longer-budget ELSA-support experiment failed under the tested QA-style recovery protocol after initially improving; this is evidence that the recipe requires redesign, not that ELSA generally fails at long budgets (Appendix B, Figure 3).

5.4 Archived Breadth and Deployment

Archived single-run evaluations cover eight model families and are summarized in Appendix D. They suggest broadly consistent but architecture-dependent transfer, so we keep the breadth claim out of the central result. Separately, archived H800 TensorRT-LLM measurements report 1.40–1.44× throughput and 0.58× weight memory for an export attributed to an earlier pipeline-generation checkpoint; these numbers do *not* correspond to any export of the checkpoint evaluated in this paper (none exists), and their provenance—the identity of the exported checkpoint—is unverifiable in this workspace, so they are reporting-only historical records rather than a new systems benchmark.

6 Limitations

- **Endpoint and uncertainty scope.** The recovery-compression result uses three archived endpoints rather than a contemporary multi-seed study. The 0.52 pp quantity is a marginal per-checkpoint binomial standard error, not pairwise or joint uncertainty or training-run variance; per-item predictions were not retained. Historical error bars use different legacy suites and do not transfer to the current harness (Appendix E).
- **Budget and mechanism scope.** No valid 625M-token matched-budget SparseForge endpoint exists, so the archived 5B comparison is not a token-normalized sample-efficiency result. The curvature score, annealing schedule, and SLoRB branch were not isolated under matched budgets, and no reported gain is attributable to one component. Curvature estimation and retraining also cost more than one-shot pruning.
- **Export and provenance scope.** Every reported SparseForge number is a branch-active training-state measurement. The SLoRB fold, exact 2:4 reprojection, group-sum assertion, and pre/post-export quality remain unimplemented or unmeasured (Appendix C, EXT-M3). The archived weights and corpus manifest are unreachable, so neither export equivalence nor corpus attribution can be resolved within this study.
- **External validity.** Cross-family results are archived single runs without recoverable per-seed or complete harness metadata. The long-budget ELSA-support recipe diverges and must be redesigned before supporting a method-level conclusion. Historical H800/TensorRT-LLM measurements come from an unverifiable earlier export, so they do not establish speed for the checkpoint studied here or transfer across hardware and workloads.

7 Conclusion

Identical 625M-token recovery compresses the registered cross-support Avg-9 point-estimate range from 2.675 to 0.118 pp. Because paired per-item predictions were not retained, the joint uncertainty and residual distinguishability of those supports remain unidentified; the result is a descriptive endpoint observation, not an equivalence or mechanism claim. The archived branch-active 5B checkpoint is supporting context only: its corpus identity is unavailable, its component effects are not isolated, and its fold to a deployable exact-2:4 export is unmeasured. Stronger method or deployment conclusions require matched-budget, multi-seed, paired, and export evidence.

Ethical considerations. SparseForge uses an additional recovery corpus during post-training (5B tokens in the archived configuration; corpus provenance unavailable, Appendix C) and therefore exposes the compressed model to recovery data whose governance cannot be audited here. Compression may reduce inference memory and time but consumes recovery compute and retains the base model's bias, privacy, licensing, and misuse risks. The reported measurements are not a life-cycle carbon assessment.

References

Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sid Black, Jordan Clive, Anthony DiPofi, Julen Etxaniz, Benjamin Fattori, Jessica Zosa Forde, Charles Foster, Mimansa Jaiswal, Wilson Y. Lee, Haonan Li, Charles Lovering, Niklas Muennighoff, Ellie Pavlick, Jason Phang, Aviya Skowron, Samson Tan, Xiangru Tang, Kevin Wang, Genta Indra Winata, Franccois Yvon, and Andy Zou. Lessons from the trenches on reproducible evaluation of language models. *ArXiv*, abs/2405.14782, 2024. URL `https://api.semanticscholar.org/CorpusID:269982020`.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. In *AAAI Conference on Artificial Intelligence*, 2019. URL `https://api.semanticscholar.org/CorpusID:208290939`.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. *ArXiv*, abs/1905.10044, 2019. URL `https://api.semanticscholar.org/CorpusID:165163607`.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *ArXiv*, abs/1803.05457, 2018. URL `https://api.semanticscholar.org/CorpusID:3922816`.

Gongfan Fang, Hongxu Yin, Saurav Muralidharan, Greg Heinrich, Jeff Pool, Jan Kautz, Pavlo Molchanov, and Xinchao Wang. Maskllm: Learnable semi-structured sparsity for large language models. *ArXiv*, abs/2409.17481, 2024. URL `https://api.semanticscholar.org/CorpusID:272910976`.

Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. *ArXiv*, abs/2301.00774, 2023. URL `https://api.semanticscholar.org/CorpusID:255372747`.

Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both weights and connections for efficient neural network. In *Neural Information Processing Systems*, 2015. URL `https://api.semanticscholar.org/CorpusID:2238772`.

Babak Hassibi and David G. Stork. Second order derivatives for network pruning: Optimal brain surgeon. In *Advances in Neural Information Processing Systems*, 1993.

Weiyu Huang, Guohao Jian, Yuezhou Hu, Jun Zhu, and Jianfei Chen. Pruning large language models with semi-structural adaptive sparse training. In *AAAI Conference on Artificial Intelligence*, 2024. URL `https://api.semanticscholar.org/CorpusID:271544038`.

Weiyu Huang, Yuezhou Hu, Jun Zhu, and Jianfei Chen. Cast: Continuous and differentiable semi-structured sparsity-aware training for large language models. *ArXiv*, abs/2509.25996, 2025. URL `https://api.semanticscholar.org/CorpusID:281682355`.

M.F. Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, 19(2):433–450, 1990. doi: 10.1080/03610919008812866. URL `https://doi.org/10.1080/03610919008812866`.

Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard H. Hovy. Race: Large-scale reading comprehension dataset from examinations. *ArXiv*, abs/1704.04683, 2017. URL `https://api.semanticscholar.org/CorpusID:6826032`.

Yann LeCun, John S. Denker, and Sara A. Solla. Optimal brain damage. In *Advances in Neural Information Processing Systems*, 1990.

Kwanhee Lee, Hyeondo Jang, Dongyeop Lee, Dan Alistarh, and Namhoon Lee. The unseen frontier: Pushing the limits of llm sparsity with surrogate-free admm. In *International Conference on Learning Representations*, 2026.

Hongyi Liu, Rajarshi Saha, Zhen Jia, Youngsuk Park, Jiaji Huang, Shoham Sabach, Yu-Xiang Wang, and George Karypis. Proxsparse: Regularized learning of semi-structured sparsity masks for pretrained llms. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

Xiang Meng, Kayhan Behdin, Haoyue Wang, and Rahul Mazumder. Alps: Improved optimization for highly sparse one-shot pruning for large language models. In *Advances in Neural Information Processing Systems*, 2024.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Conference on Empirical Methods in Natural Language Processing*, 2018. URL `https://api.semanticscholar.org/CorpusID:52183757`.

NVIDIA. Nvidia ampere architecture in-depth. `https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/`, 2020. Accessed: 2026-04-30.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande. *Communications of the ACM*, 64:99 – 106, 2019. URL `https://api.semanticscholar.org/CorpusID:198893658`.

Sidak Pal Singh and Dan Alistarh. Woodfisher: Efficient second-order approximations for model compression. *ArXiv*, abs/2004.14340, 2020. URL `https://api.semanticscholar.org/CorpusID:216641895`.

Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models. *ArXiv*, abs/2306.11695, 2023. URL `https://api.semanticscholar.org/CorpusID:259203115`.

Hugo Touvron, Louis Martin, Kevin R. Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Niko lay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Daniel M. Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony S. Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel M. Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, R. Subramanian, Xia Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zhengxu Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melissa Hall Melanie Kambadur, Sharan Narang, Aur'elien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *ArXiv*, abs/2307.09288, 2023. URL `https://api.semanticscholar.org/CorpusID:259950998`.

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems. *ArXiv*, abs/1905.00537, 2019. URL `https://api.semanticscholar.org/CorpusID:143424870`.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Annual Meeting of the Association for Computational Linguistics*, 2019. URL `https://api.semanticscholar.org/CorpusID:159041722`.

A Statistical Appendix: Avg-9 Uncertainty and Recovery Compression

A.1 Sampling uncertainty of the archived SparseForge vs. AST margin

We quantify the uncertainty of the archived comparison SparseForge − AST (Avg-9, Table 1) using only the registered same-harness per-task accuracies of `cast9_dense_ast_current_harness.csv` and the standard lm-eval dev-set sizes n_k of the nine tasks (BoolQ, RTE, HellaSwag, WinoGrande, ARC-E, ARC-C, OBQA, RACE, PIQA), which Table 3 lists task by task.

Leave-one-out (LOO-9). The full nine-task margin is $\Delta_{\text{Avg}-9} = +0.528$ pp. Table 3 reports, for each of the nine tasks dropped one at a time, the remaining eight-task margin, $\Delta^{(k)}_{\text{Avg}-8}$, and its deviation $\delta_k = \Delta^{(k)}_{\text{Avg}-8} - \Delta_{\text{Avg}-9}$ from the full mean.

Table 3: Leave-one-task-out sensitivity of the Avg-9 margin. n_k is the standard lm-eval dev-set size that enters the binomial standard errors. Dropping the smallest task (OpenBookQA, n=500) reverses the sign of the remaining eight-task margin to −0.056 pp; dropping ARC-Challenge or BoolQ collapses it to ≈0.05–0.10 pp; only dropping the AST-side RTE deficit (RTE = SparseForge 49.82 vs. AST 65.34) inflates the margin to +2.535 pp (δ=+2.006). The archived +0.528 pp margin is not robust to single-task removal.

Dropped task	n_k	k-drop Avg-8 margin (pp)	δ_k vs full (pp)
OpenBookQA	500	−0.056	−0.584
ARC-Challenge	1,172	+0.050	−0.478
BoolQ	3,270	+0.097	−0.431
ARC-Easy	2,376	+0.215	−0.313
RACE	1,045	+0.247	−0.281
WinoGrande	1,267	+0.387	−0.141
HellaSwag	10,042	+0.602	+0.074
PIQA	1,838	+0.676	+0.148
RTE	277	+2.535	+2.006

Avg-9 sampling error. Treating each checkpoint as fixed and each task accuracy as binomial, the sampling SE of an unweighted nine-task mean is $\text{SE} = (1/9)\sqrt{\sum_k p_k(1-p_k)/n_k}$, giving $\text{SE}_{\text{SF}} = 0.52$ pp and $\text{SE}_{\text{AST}} = 0.51$ pp per checkpoint. The resulting difference SE and the z and two-sided p of the archived margin are the aggregate row of Table 4: the observed margin lies inside sampling noise.

Per-task tests. McNemar omitted: per-item logs are not available in the registered artifacts. Exact per-example McNemar would require the discordant-pair counts $b = \#(\text{SF right, AST wrong})$ and $c = \#(\text{SF wrong, AST right})$ per task; these are provably *unidentifiable* from the registered aggregate CSV, which records only the marginal correct counts (SF_correct= $a+b$, AST_correct= $a+c$; only $b-c$ is fixed, and infinitely many (a,b,c,d) decompositions meet both margins). Per-item predictions for these nine tasks are not available in the registered artifacts. The strongest test computable from the registered data is therefore the unpaired normal-approximation z-test on the difference of two independent binomial proportions (equal n in both arms). Table 4 collects every test statistic that the registered data support. At the uncorrected 5% level, four tasks are significant (BoolQ, RTE, ARC-Easy, ARC-Challenge); under Holm correction across nine tasks, only BoolQ and RTE remain significant.

Boundary. A per-example paired (McNemar) analysis would be the decisive tighter test, but it requires per-item predictions that are not in the registered set in the registered artifacts; it is gated on a future registered run producing per-item inference outputs, and is not computable from the aggregate CSV. This boundary is stated verbatim in the machine-readable delivery (`stat_delivery_v2.json`).

Table 4: Significance tests computable from the registered aggregate CSV; Δ is SparseForge − AST. All per-task rows use the same unpooled two-proportion z convention (equal n in both arms); SE is the sampling SE of the accuracy difference. No per-item logs exist, so none of these is a paired test. Under the uncorrected 5% convention four task-level pairs reach significance (BoolQ, RTE, ARC-Easy, ARC-Challenge); under Holm correction across the nine tasks only BoolQ and RTE do. The pooled count-level row and the Avg-9 row are reported as in the earlier revision. The final column reports uncorrected/Holm significance.

Comparison (test)	Δ	SE	z	p	p_{Holm}	Sig. raw/Holm
BoolQ	+3.97	1.05	+3.79	0.0002	0.0013	yes/yes
RTE	−15.52	4.20	−3.70	0.0002	0.0015	yes/yes
HellaSwag	−0.06	0.70	−0.08	0.932	1.0000	no/no
WinoGrande	+1.66	1.84	+0.90	0.367	1.0000	no/no
ARC-Easy	+3.03	1.26	+2.41	0.016	0.1118	yes/no
ARC-Challenge	+4.35	2.04	+2.14	0.033	0.1948	yes/no
OpenBookQA	+5.20	2.97	+1.75	0.079	0.3948	no/no
RACE	+2.78	2.15	+1.29	0.196	0.7825	no/no
PIQA	−0.66	1.40	−0.47	0.640	1.0000	no/no
Nine tasks (pooled count z)	+1.23	0.47	+2.63	0.008	–	–
Nine tasks (6/9 wins, exact sign)	—	—	—	0.508	–	–
Avg-9 margin (checkpoint z)	+0.528	0.73	+0.72	0.47	–	–

A.2 Recovery compresses the cross-support spread below the sampling-noise scale

Table 5 reports native and identical 625M-token joint weight+SLoRB recovery results from `trainingfree_lowtraining_detailed_ast7.csv`. The post-recovery range $R(T) = 0.118$ pp is far below the 0.52 pp per-checkpoint binomial-SE sampling-noise scale[3]. Its joint uncertainty is unidentified from the registered aggregates, so we do not quote a compression ratio.

Table 5: Recovery compresses the cross-support range from 2.675 pp to 0.118 pp, far below the 0.52 pp per-checkpoint binomial-SE sampling-noise scale (§A.1). This is a descriptive marginal-scale comparison; the aggregates do not identify paired or joint uncertainty. The weaker native supports gain most and the ALPS↔ProxSparse rank flips.

Support	native AVG-9 (0 tok)	recovered AVG-9 (625M)	gain
ALPS	52.12	55.82	+3.70
ELSA-4096	54.59	55.94	+1.34
ProxSparse	51.92	55.91	+3.99
range	2.675	0.118	

Conjecture (C1): annealed-support signal survives compression only via an asymptotic intercept. The following is a *conjectured* regime statement, not a proved theorem: it is displayed because an external soundness review exhibited an explicit counterexample to the corresponding unconditioned claim (one can place the observed range $R(T)$ into the recovery-invariant subspace while keeping the transients active, so the "iff" fails at finite T), and*no* stronger bounded-T claim is made in this revision. We state the conjectured repair so the reader sees our best current mechanism explanation and the assumptions that would have to hold for it to be adequate. Let $A_s(0)$ and $A_s(T)$ be the AVG-9 of support s before and after T recovery tokens, and let $\phi_s = \lim_{T\to\infty} A_s(T)$ be the recovery attractor; assume the intercepts exist (conjectured regime, not established). Suppose $A_s(T) = \phi_s + \big(A_s(0) - \phi_s\big)\, g_s(T)$ with $g_s(T) \to 0$ in a recovery-dominated regime (T large enough that the transient term is uniformly dominated). Then the cross-support range $R(T) = \max_s A_s(T) - \min_s A_s(T)$ converges to $\lim_{T\to\infty} R(T) = \max_s \phi_s - \min_s \phi_s$. If the supports share one attractor ($\phi_s = \phi_{s'}$ for all s, s'), then for any T with $g_s(T) \approx g_{s'}(T)$ the native signal is erased: $A_s(T) - A_{s'}(T) \approx 0$ up to recovery noise, and rank orders that depend only on the transient $A_s(0) - \phi_s$ are not recoverable. Under the conjectured regime above (regime rate condition: the

[3]The 0.52 pp term is the per-checkpoint Avg-9 binomial-SE; pairwise and joint (max-min) uncertainty involve covariance and selection effects not captured by a single marginal SE, and require per-item predictions not contained in the registered aggregate CSVs

transient $g_s(T)$ contribution is dominated by the intercept contribution to $R(T)$) we *conjecture* that a support/mask signal survives recovery-compression iff it is carried by an *asymptotic intercept* $\phi_s - \phi_{s'} \neq 0$ that (i) is monotone under the recovery-preserving transformation and (ii) is not absorbed into a shared fixed point, i.e. the advantage lies in the recovery-invariant subspace. The two-line calculation that $R(T) \to \max_s \phi_s - \min_s \phi_s$ is the only step we can justify in this revision. The existence and joint uncertainty of distinct intercepts relative to the 0.52 pp per-checkpoint binomial-SE sampling-noise scale, and the recovery-invariant-subspace statement, remain open.

Implication (descriptive). SparseForge's +0.53 pp over AST uses 5B recovery tokens, an 8× larger budget than the 625M rows in Table 5, where the cross-support range is far below the 0.52 pp binomial-SE sampling-noise scale. Because the conjectured regime conditions and paired/joint uncertainty are unverified, we draw no conclusion beyond budget confounding. No registered experiment measures SparseForge's own support at 625M (the resolving row), so the +0.53 pp claim remains budget-confounded and is not yet attributable to the mask mechanism.

B ELSA Fixed-Support Long-Budget Failure Mode

Figure 3 shows that the longer-budget ELSA-support curve improves, then destabilizes, then diverges; its caption carries the registered per-step values. This is the only on-disk experiment with complete per-run raw artifacts; its divergence (not a mechanism-level failure of ELSA as a method) is why Sec. 6 frames the failure as "the longer-budget recipe requires redesign, not that ELSA generally fails."

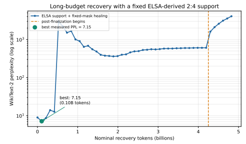

Figure 3: Registered recovery trajectory of the fixed ELSA support on the 5B-token SparseForge recovery stream: PPL drops from 8.99 (step 0) to a minimum of 7.15 (step 100), destabilizes step 300 → 500, and diverges past step 1000 (2930 at step 500, 4083 at step 4600). Raw per-step values are included in the anonymous artifact.

C Implementation Details and Artifact Statement

This appendix is the executable specification of the learned-mask path at the frozen code commit `520170b9`. It separates that algorithm from the archived fixed-support recovery launchers: the latter set `change_mask=False`, so their mask, temperature, structural-mixing, and hardening schedules are inactive. Table 6 collects the operating constants of those reported recovery arms; the paragraphs after the table specify the active learned-mask algorithm and its export boundary.

Table 6: Hyperparameters and protocol constants of the reported SparseForge recovery arms. "625M" rows are the three fixed-support recovery endpoints; "5B" is the single archived configuration used as supporting context. A dash marks a quantity not fixed by the archived run protocol. Unspanned rows give the two arms separately in the two value columns; a value spanning both columns applies to the 625M recovery arm and the 5B strong configuration alike.

Quantity	625M recovery rows	5B strong configuration
Optimizer	custom AdamW	custom AdamW
Adam betas	β_1=0.9, β_2=0.95	
Fused optimizer kernel	no	no
Base random seed	0	0
Sequence length	2048	4096
Global batch size (seq)	256	256
Micro batch size	1	8
Total steps	1192	4768 (= 4053+715)
Peak step size	2×10^{-5}	1×10^{-4}
Final step size	2×10^{-6}	1×10^{-5}
Step-decay schedule	cosine	cosine
Warmup steps	50	477
Step-weight decay	0	0.1
$\lambda_{\text{KL}} : \lambda_{\text{task}}$	1:1 (CE + KL)	
KL teacher	the pre-finetune dense checkpoint itself	
Distillation temperature	2.0	code default[4]
Mask-temperature T (initial)	0.7 (launcher record)	1.0 (code default; manifest records no override[5])
SLoRB rank / init	16 / zero-init	
Hardening start (step)	inactive	2861
Hardening duration (steps)	inactive	1192
Mask-switch step	477	477
Post-finalization finetune	disabled	715
Transformers	`4.57.6`	
lm-eval-harness	≥ 0.4.0	

C.1 Executable alternating weight/mask algorithm

For every sparse linear layer ℓ, let $W_\ell, m_\ell, H_\ell \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$, with d_{in} divisible by $M = 4$. The code views each row as $d_{\text{in}}/4$ consecutive groups, hence grouped tensors have shape $d_{\text{out}} \times (d_{\text{in}}/4) \times 4$, and retains $N = 2$ indices in the final hard mask. The SLoRB forward branch is $B_\ell P_\ell$, implemented as two linear maps after the masked base-weight map; P_ℓ is fixed unless `trainable_projection=True`.

Algorithm 1: alternating weight and mask tracks.

Input: training batches; W_ℓ; continuous masks $m_\ell \in [0,1]^{d_{\text{out}}\times d_{\text{in}}}$; SLoRB factors (B_ℓ, P_ℓ); update period K; and schedules below.

1. For optimizer step t and each gradient-accumulation micro-step, compute each sparse-linear output with $m_{\text{eff},\ell}$ and the SLoRB add-on, then form $\mathcal{L}_{\text{train}} = \lambda_{\text{task}}\text{CE} + \lambda_{\text{KL}}\text{KL}$ (both coefficients are 1 in the reported distillation recipe).
2. On the last micro-step only, if `enable_hutchinson=True`, draw one independent Rademacher tensor per W_ℓ and update H_ℓ by the double-backward rule in the next subsection. The differentiated parameter list contains the sparse base weights W_ℓ only, not masks or SLoRB factors.
3. Back-propagate the accumulated loss and take one AdamW weight-track step on the trainable parameters. The mask is a non-autograd parameter and is not changed by this optimizer step.

[5]The portable 625M launcher pins a distillation temperature of 2.0. The archived 5B launch manifest does not record a temperature override, so the trainer's code default applies. We report this distinction explicitly rather than silently substitute one default for the other. The frozen LLaMA path imports the repository-local `legacy/adamw.py`; it does not pass a `fused` flag or dispatch to PyTorch's fused AdamW kernel.

[5]The archived 5B launch manifest records no mask-temperature override, so the trainer's code default 1.0 applies on the 5B side; the 0.7 record in the portable 625M launcher is reported literally to its left.

Algorithm 1 (continued): mask track and finalization.

4. If t is before `sparsity_warmup_steps`, or $t \bmod K \neq 0$, skip the mask track. Otherwise, using the post-optimizer W_ℓ, compute the standardized Hessian–OBD scores, groupwise gate and exact top-2 target, structural mixture, 2:4 penalty pull, and mask EMA specified below. Update the stored hardening coefficient h_x on this same mask update.
5. After the final step, construct the hard top-2 mask, apply the implemented state-dict finalization, run the exact support assertion below (per-group sum = 2; specified as the required export unit check but not wired into the frozen validator, Appendix C), and save the checkpoint. The distinct SLoRB-fold/re-projection operation is not implemented by this finalizer and is therefore a separate future export measurement, not an executed line of Algorithm 1.

C.2 Hutchinson diagonal on the training loss

On the last micro-step of every enabled optimizer update, the implementation passes the (gradient-accumulation-scaled) training loss to `update_hessian_hutchinson`. Terms depending only on the detached mask do not alter its derivatives with respect to W. For every sparse base weight and one fresh probe $v_\ell \sim \text{Rad}(\{-1,+1\})^{W_\ell}$, it computes

$$g_\ell = \nabla_{W_\ell}\mathcal{L}_{\text{train}}, \qquad u = \sum_\ell \langle g_\ell, v_\ell\rangle, \qquad \widehat{H}_\ell = \max(0, \nabla_{W_\ell} u \odot v_\ell)\,. \tag{7}$$

The first `torch.autograd.grad` uses `create_graph=True`; the second differentiates u with respect to the same W-only list. Thus this is one Hutchinson sample per enabled update, not one sample per mask update and not an average over accumulation micro-steps. In DDP, $\widehat{H}_\ell$ is all-reduced and divided by the world size before the EMA $H_\ell \leftarrow 0.99H_\ell + 0.01\widehat{H}_\ell$; 0.99 is `cfg.beta`. The frozen FSDP branch instead updates local shards without that DDP all-reduce, a distinction required to reproduce the code exactly.

C.3 Score, gate, schedules, and mask recurrence

The `hessian_obd` metric forms $S^{\text{raw}}_\ell = (H_\ell + 10^{-8}) \odot W^2_\ell$ and standardizes it over the complete layer,

$$S_\ell = \frac{S^{\text{raw}}_\ell - \mu(S^{\text{raw}}_\ell)}{\text{std}(S^{\text{raw}}_\ell) + 10^{-6}}. \tag{8}$$

After reshaping to $d_{\text{out}} \times (d_{\text{in}}/4) \times 4$, let I_g be the two indices returned by `torch.topk` and let $\bar{G}_g$ be zeros with ones scattered at I_g. In the soft structured path, the implemented threshold is the second-largest score $\tau_g = S_{g,(2)}$ (not the midpoint of the second- and third-largest scores), and $G^{\text{soft}}_g = \sigma((S_g - \tau_g)/(T + 10^{-8}))$. If `structured_exact=True`, the code bypasses this sigmoid and returns $\bar{G}_g$ directly.

With K = `mask_update_period` = 10, the default temperature schedule is

$$T(t) = \max\Big(0.05,\ 1.0 \times 0.97^{\lfloor t/K \rfloor}\Big), \tag{9}$$

corresponding to `temp_init=1.0`, `temp_min=0.05`, and `temp_decay=0.97`. Mask updates occur only when $t \geq$ `sparsity_warmup_steps` and $t \bmod K = 0$. For configured structural endpoints $t_s < t_e$, set $q = \text{clip}((t - t_s)/(t_e - t_s), 0, 1)$ and $\beta(t) = 3q^2 - 2q^3$; then $G^\beta = (1-\beta)G^{\text{soft}} + \beta\bar{G}$. A disabled interval ($t_e \leq t_s$) yields $\beta = 0$.

For `mask_penalty_mode=nm_2_4`, the code takes another per-group top-2 target $\bar{G}^m$ from the current gate ordering and applies the no-grad pull

$$\begin{aligned} G' &= \text{clip}_{[0,1]}\big[G^\beta - (\texttt{pen_scale}\,\lambda_{\text{mid}})(m - \bar{G}^m)\big]\,, \\ m &\leftarrow (1-a)m + aG', \qquad a = \texttt{mask_lr} = 0.10. \end{aligned} \tag{10}$$

Here `pen_scale` is `mask_penalty_lr`, falling back to `mask_lr` when unset; in the non-block modes λ_{mid} ramps linearly from zero to its configured maximum over the first 500 optimizer steps and then remains constant.

For hardening start t_h and duration D, the stored coefficient is $h_x = 1$ before t_h, $h_x = 1-(t-t_h)/D$ on the interval, and $h_x = 0$ at and after $t_h + D$ (clipped to $[0,1]$ and refreshed on mask-update steps). The forward pass uses

$$m_{\text{eff}} = h_x m + (1-h_x)\bar{G}^m. \tag{11}$$

If CAST-style `weight_scaling` is enabled, its 2:4 target factor is $M/N = 2$ and the actually applied factor is $c(h_x) = 1 + (1-h_x)(2-1)$; it therefore anneals from 1 to 2 with the same h_x. The reported fixed-support launcher disables this option.

C.4 Tie rule and exact-2:4 validation

Every structured selection uses the indices returned by `torch.topk(..., largest=True)` and creates the binary target by `scatter_`. This guarantees two written ones per group for a returned index tuple, including the `structured_exact` path, but the frozen code adds no deterministic secondary key. PyTorch/CUDA does not expose a stable mathematical ordering for equal scores, so tied groups have an underspecified support: the kernel's returned indices decide the survivors, and a unique cross-device tie rule is not guaranteed.

The required export unit assertion is therefore the groupwise, not merely global, check

```
assert torch.all(hard.view(d_out,d_in//4,4).sum(-1) == 2)
```

At the frozen commit, `legacy/check_sparsity.py:57-121,137-181` checks binary/global mask sparsity and mask–weight zero overlap, but contains *no* group-sum-= 2 assertion. It cannot honestly be cited as that exact unit test until the assertion above is added to the export validator. During training, `nm_2_4_tile_stats` (`legacy/utils.py:1029-1234`) reports top-2/bottom-2 mask means and soft–hard gaps; these are useful trajectory diagnostics but do not replace the exact export assertion.

C.5 Finalization, SLoRB fold boundary, and EXT-M3

The algebraic fold needed to remove the inference branch would first form $W^{\text{fold}}_\ell = W_\ell + B_\ell P_\ell$ and then re-project $W^{\text{export}}_\ell = \Pi_{2:4}(W^{\text{fold}}_\ell)$. The frozen `main_llama.py` finalizer does less: it recomputes a hard mask from the soft mask, writes that mask to the state dict, replaces W_ℓ by $W_\ell \odot \bar{G}_\ell$, and saves `model.pt`. It does not add $B_\ell P_\ell$ into W_ℓ in this path. Consequently, neither the fold-then-2:4 re-projection delta nor pre/post-export WikiText-2 PPL, Avg-9, or support-change has been measured. We register those four checks as future measurement **EXT-M3**; no deployment-equivalence claim is inferred from the training checkpoint.

C.6 Optimizer and software environment

The frozen LLaMA path uses the repository-local AdamW implementation with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1 on matrix parameters and zero decay on lower-dimensional parameters. The archived launch manifest pins base seed 0; DDP adds the rank to that seed. This optimizer is Python/tensor AdamW rather than a fused optimizer kernel. The committed environment contract pins `transformers==4.57.6` and requires `lm_eval>=0.4.0`; the latter is a lower bound rather than an exact version, so a bitwise reproduction must additionally record the resolved environment.

Artifact statement. The training entry point is the repository's legacy `main_llama.py`; fixed-support launchers are included in the anonymous artifact. Evaluation uses lm-evaluation-harness in the local wrapper `eval_native_checkpoint_shard.py`. The recovery corpus of the archived 5B checkpoint is not named in this paper: that corpus is recorded in a training manifest (`args.json`) archived with the checkpoint on an archive host unreachable from this workspace since 2026-08-16, so the identity of the archived recovery corpus cannot be verified from available materials. The modern matched-budget controls under preparation (Appendix A, Table 1 caption) use the locally tokenized Dolmino-mix-1124 stream instead. The identifying repository URL is replaced for double-blind review by the anonymized mirror `https://anonymous.4open.science/r/SparseForge-Anonymous/`.

C.7 Archived-checkpoint unreachability disclosure

The 5B headline checkpoint, its training manifest (`args.json`), and its recovery-corpus tokenization live on an archive host that is unreachable from this workspace; independent re-scans of local storage returned no copy. Consequently: (i) the archived recovery corpus is not named anywhere in this paper — earlier revisions' conflicting attributions (a Dolmino-style mix in the setup paragraph; a QA-format SFT configuration in the artifact statement) cannot be disambiguated from available materials and have been withdrawn; (ii) fold/re-projection would require the archived branch parameters and is registered as future measurement (EXT-M3); and (iii) every reported SparseForge number in this paper remains a branch-active training-state measurement whose corpus provenance and export correspondence are unresolved. We disclose, and the record does not allow us to resolve either dependency from the materials at hand.

D Archived Cross-Family Evidence

Table 7 preserves archived single-run block-16 results as model-family breadth evidence only.

Table 7: Archived single-run block-16 mean accuracy. A registered-pack grep found no headline-value hits, so the unrecoverable seven-task suite is not named. All rows are English-only; the axis is model family, not language. Diff is sparse minus dense. Transfer is broadly consistent but architecture-dependent, with Qwen3-1.7B the largest degradation; these are breadth records, not a controlled cross-family claim.

Model	Dense	Sparse	Diff
LLaMA-2-7B	59.70	59.02	−0.68
OPT-2.7B	47.76	47.93	+0.17
GPT-2	37.13	37.58	+0.45
GPT-2-Medium	41.01	39.45	−1.56
GPT-2-Large	42.65	41.87	−0.81
GPT-2-XL	45.49	45.14	−0.35
Qwen3-1.7B	56.47	53.49	−2.98
DeepSeek-MoE-16B	59.54	58.20	−1.34

E Historical Training-Run Variability

Table 8 preserves author-confirmed three-run aggregates; missing run mappings and different legacy suites make the rows mutually and currently incomparable.

Table 8: Historical three-run aggregates (mean ± sample SD). Corpus labels are unverified contemporaneous records; because the rows use different legacy suites, neither is comparable with the other or the current harness.

Setting	Legacy suite	Accuracy	Wiki PPL
5B Dolmino	CAST-7	57.27 ± 0.09	6.22 ± 0.04
1.25B C4	AST-7	57.23 ± 0.13	6.38 ± 0.03

F Baseline Dispersion

Table 9 reports sample SDs from three independent baseline seeds where available (accuracy to two decimals, PPL to four); Table 1 gives the means.

Table 9: Three-seed sample SD under the unified harness. Aggregate SD appears only for paired per-seed aggregates; a dash means it is not inferred from marginal task statistics.

Method	BQ	RTE	Hella.	Wino.	ARC-e	ARC-c	OBQA	RACE	PIQA	Avg-9	PPL
Wanda	0.14	0.36	0.12	0.08	0.49	0.48	0.31	0.15	0.26	0.13	0.0525
SparseGPT	1.29	1.65	0.09	0.72	0.86	0.26	0.92	0.19	0.25	0.36	0.0103
ALPS	1.34	0.36	0.18	0.28	0.68	0.85	0.70	0.64	0.24	–	0.0896
ELSA (4,096 steps)	1.06	2.54	0.06	0.83	0.56	0.53	0.64	1.23	0.17	–	0.0168